

Predicting cellular abundances of ciliates from ribosomal RNA gene copy numbers using digital PCR

Megan Gross, Micah Dunthorn, Quentin Mauvisseau, Thorsten Stoeck

2023-11-17

LIBRARIES

```
library("tidyverse")
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.3      v readr      2.1.4
## v forcats    1.0.0      v stringr   1.5.0
## v ggplot2    3.4.3      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.0
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library("knitr")
library("ggpubr")
library("rstatix")
```

```
##
## Attaching package: 'rstatix'
##
## The following object is masked from 'package:stats':
##
##     filter
```

```
library("vegan")
```

```
## Loading required package: permute
## Loading required package: lattice
## This is vegan 2.6-4
```

```
library("scales")
```

```
##
## Attaching package: 'scales'
##
## The following object is masked from 'package:purrr':
##
##     discard
##
## The following object is masked from 'package:readr':
##
##     col_factor
```

DATASET

```
read.table(
  "df_precision.csv",
  head = T,
  sep = ";",
  stringsAsFactors = F,
  check.names = F
) -> df_precision

read.table(
  "copynumber_per_cell.csv",
  head = T,
  sep = ";",
  stringsAsFactors = F,
  check.names = F,
  na.strings = c("", "NA")
) -> df_cn_cell

read.table(
  "copynumber_growthphase_all_cells_changed.csv",
  head = T,
  sep = ";",
  stringsAsFactors = F,
  check.names = F
) -> df_gp_all

read.table(
  "dPCR_comparison_primers_48.29RFU.csv",
  head = T,
  sep = ";",
  stringsAsFactors = F,
  check.names = F
) -> df_primers

read.table(
  "dPCR_cellratio.csv",
  head = T,
  sep = ";",
  stringsAsFactors = F,
```

```

    check.names = F
) -> df_ratio_reg

read.table(
  "scatterplot_data.csv",
  head = T,
  sep = ",",
  stringsAsFactors = F,
  check.names = F
) -> df_scatter

read.table(
  "scatterplot_data_all.csv",
  head = T,
  sep = ",",
  stringsAsFactors = F,
  check.names = F
) -> df_scatter_all

# set n seeds accordingly to make the script reproducible
set.seed(1837)

```

THEME & COLOR PALETTES

```

c("#660000",
  "#003300",
  "#996600") -> palette_1

c("cornflowerblue",
  "darkblue") -> palette_2

c("#660000",
  "salmon3") -> palette_3

theme(
  panel.grid.major.x = element_line(size = 0.5, colour = "#f0f0f0"),
  panel.grid.major.y = element_blank(),
  panel.grid.minor = element_blank(),
  panel.background = element_blank(),
  panel.border = element_rect(
    colour = "black",
    fill = NA,
    size = 1
  ),
  axis.line = element_line(colour = "black"),
  legend.position = ("none"),
  legend.key = element_blank(),
  axis.text = element_text(colour = "black", size = 14),
  strip.placement = "outside",

```

```

strip.text = element_text(size = 14),
axis.title.x = element_text(size = 16, vjust = -2),
axis.title.y = element_text(
  size = 16,
  angle = 90,
  vjust = 5
),
plot.margin = unit(c(1, 0.5, 1, 1), "cm")
) -> my_theme

theme(
  panel.grid.major.x = element_line(size = 0.5, colour = "#f0f0f0"),
  panel.grid.major.y = element_blank(),
  panel.grid.minor = element_blank(),
  panel.background = element_blank(),
  panel.border = element_rect(
    colour = "black",
    fill = NA,
    size = 1
  ),
  axis.line = element_line(colour = "black"),
  legend.position = c(.75, .32),
  legend.background = element_rect(color = "black", linetype = "solid"),
  legend.title = element_text(size = 18),
  legend.text = element_text(size = 16),
  legend.justification = c("right", "bottom"),
  legend.box.just = "right",
  legend.margin = margin(6, 6, 6, 6),
  axis.text.x = element_blank(),
  axis.text.y = element_text(colour = "black", size = 14),
  strip.placement = "outside",
  strip.text = element_text(size = 14),
  axis.title.x = element_blank(),
  axis.title.y = element_text(
    size = 20,
    angle = 90,
    vjust = 5
  ),
  plot.margin = unit(c(1, 0.5, 1, 1), "cm")
) -> my_theme_gp

```

GROWTH CURVES

```

cell_counts <-
  read.table(
    "cell_counts.csv",
    head = T,
    sep = ";",
    stringsAsFactors = F,
    check.names = F
  )

```

```

)

cell_counts$day <- as.character(cell_counts$day)
ggplot(cell_counts,
  aes(
    x = day,
    y = mean_of_replicates,
    group = Culture,
    color = Culture
  )) +
  geom_errorbar(
    aes(ymin = mean_of_replicates - sd, ymax = mean_of_replicates + sd),
    width = .1,
    size = 1
  ) +
  geom_line(lwd = 1.3) +
  geom_vline(
    xintercept = c(4, 5, 6, 8),
    color = "black",
    size = 1,
    linetype = "dashed"
  ) +
  geom_point(size = 2) +
  scale_color_brewer(palette = "Paired") +
  xlab("Day") +
  ylab("Cell density (ind. per ml)") +
  my_theme +
  theme(legend.position = ("right")) -> growth_curve

```

MEAN COPY NUMBER PER CELL

```

#compute summary statistics
df_cn_cell %>%
  rstatix::get_summary_stats(CN_mean) -> cn_sum

#for technical replicates separately
#compute summary statistics
df_cn_cell %>%
  rstatix::get_summary_stats(CN_reps) -> cn_reps_sum

#plot histogram to inspect density distribution.
ggpubr::gghistogram(
  df_cn_cell,
  x = "CN_reps",
  y = "..density..",
  fill = "steelblue",
  bins = 4,
  add_density = TRUE
) -> histogram_plot

```

REPRODUCIBILITY AND PRECISION

```
#Test for normality
rstatix::shapiro_test(df_precision$CV) -> normality_precision

#Test for homogeneity of variance
rstatix::levene_test(df_precision, CV ~ sample) -> variance_precision

#Testing for significant difference between growth phases
df_precision %>%
  group_by(group) %>%
  rstatix::pairwise_wilcox_test(CV ~ sample) -> wilcox_precision

df_precision %>%
  group_by(group) %>%
  rstatix::wilcox_effsize(CV ~ sample) -> wilcox_effectsize

left_join(wilcox_precision,
          wilcox_effectsize,
          by = c("group1", "group2", ".y.")) -> wilcox_combine

write.csv(wilcox_combine, "wilcox_precision.csv")
```

COMPARING COPYNUMBER AND GROWTH PHASE STATISTICS

```
#compute summary statistics
df_gp_all %>%
  group_by(phase) %>%
  get_summary_stats(copynumber) -> t

#Test for normality
rstatix::shapiro_test(df_gp_all$copynumber) -> normality

#test for homogeneity of variance
rstatix::levene_test(df_gp_all, copynumber ~ phase) -> variance

gghistogram(
  df_gp_all,
  x = "copynumber",
  y = "..density..",
  fill = "steelblue",
  bins = 4,
  add_density = TRUE
) -> histogram_plot
```

```

#Testing for significant difference between growth phases
df_gp_all %>%
  rstatix::wilcox_test(copynumber ~ phase, alternative = "two.sided") -> wilcox_culture

#Check effect size
df_gp_all %>%
  rstatix::wilcox_effsize(copynumber ~ phase) -> wilcox_effectsize

left_join(wilcox_culture,
          wilcox_effectsize,
          by = c("group1", "group2", ".y.")) -> wilcox_combine

write.csv(wilcox_combine, "wilcox_gp.csv")

```

```

#Testing effect of cultures
#Checking for significant difference between cultures
df_gp_all %>%
  rstatix::wilcox_test(copynumber ~ culture, alternative = "two.sided") -> wilcox_culture

#Check effect size
df_gp_all %>%
  rstatix::wilcox_effsize(copynumber ~ culture) -> wilcox_effectsize

left_join(wilcox_culture,
          wilcox_effectsize,
          by = c("group1", "group2", ".y.")) -> wilcox_combine

write.csv(wilcox_combine, "wilcox_cultures.csv")

```

DATA VISUALIZATION

```

my_comparisons <-
  list(c("late exponential", "stationary"),
       c("early exponential", "stationary"))

#For different growth phases
df_gp_all %>%
  ggplot(aes(x = phase, y = copynumber, fill = phase)) +
  geom_point(position = position_jitterdodge(jitter.width = 0.1)) +
  geom_boxplot(aes(fill = phase), outlier.shape = NA, alpha = 0.7) +
  scale_fill_manual(values = palette_1) +
  xlab("growth phase") +
  ylab("rRNA gene copy number per cell") +
  scale_y_continuous(labels = comma, limits = c(1000, 130000)) +
  my_theme +

```

```

stat_compare_means(hjust = 0.7, label.y = 140000) +
stat_compare_means(comparisons = my_comparisons,
                    method = "wilcox.test",
                    label = "p.signif") -> cn_gf_all

#split by phase and culture

ggboxplot(df_gp_all,
          x = "phase",
          y = "copynumber",
          fill = "phase") +
  ylab("rRNA gene copy number per cell") +
  scale_fill_manual(values = palette_1) +
  scale_y_continuous(labels = comma) +
  my_theme_gp +
  labs(fill = "growth phase") +
  facet_wrap(vars(culture), nrow = 2) +
  stat_compare_means(hjust = 0.5, label.y = 140000) -> cn_gf_sep

```

OUTPUT DATA

COMPARISON COPY NUMBER OF JUST DIVIDED AND NON-DIVIDING

STATISTICS

```

#Compute summary statistics
df_cn_cell %>%
  group_by(state) %>%
  get_summary_stats(CN_mean) -> statistics_dividers

#Test for normality
rstatix::shapiro_test(df_cn_cell$CN_mean) -> normality

#Test for homogeneity of variance
rstatix::levene_test(df_cn_cell, CN_mean ~ state) -> variance

#Testing for significant difference between groups
df_cn_cell %>%
  rstatix::wilcox_test(CN_mean ~ state,
                      alternative = "two.sided") -> wilcox_state

#Check effect size
df_cn_cell %>%
  rstatix::wilcox_effsize(CN_mean ~ state) -> wilcox_effectsize

```

```

left_join(wilcox_state,
          wilcox_effectsize,
          by = c("group1", "group2", ".y.")) -> wilcox_combine

wilcox_combine

write.csv(wilcox_combine, "wilcox_state.csv")

```

DATA VISUALIZATION

```

my_comparisons <- list(c("just divided cells", "not divided cells"))

df_cn_cell %>%
  ggplot(aes(x = state, y = CN_mean, fill = state)) +
  geom_point(position = position_jitterdodge(jitter.width = 0.1), na.rm = T) +
  geom_boxplot(
    aes(fill = state),
    outlier.shape = NA,
    alpha = 0.9,
    na.rm = T
  ) +
  scale_fill_manual(values = palette_2) +
  scale_y_continuous(labels = comma, limits = c(10000, 160000)) +
  xlab("cell state") +
  ylab("rRNA gene copy number per cell") +
  my_theme +
  stat_compare_means(comparisons = my_comparisons,
                    method = "wilcox.test",
                    label = "p.signif") -> cn_cell_state

```

##SIGNIFICANCE TEST FOR SUBSET

```

#Testing 15 random non-dividing cells 20 times

df1<- df_cn_cell %>% filter(state == "not divided cells")
df2<- df_cn_cell %>% filter(state == "just divided cells", na.rm = TRUE)

df2 <- df2 %>% na.omit()
#Test for normality
rstatix::shapiro_test(df2$CN_mean) -> shapiro_df2

#Set number of repeats
choose_wdh <- 20

seeds <- sample(1:1000, choose_wdh)

#create dataframe which can be filled with results
result_matrix <- matrix(nrow = choose_wdh, ncol = 4)
colnames(result_matrix) <- c("shapiro_ts", "shapiro_p", "wilcox_p", "wilcox_ts")

```

```

#loop

for (i in 1:length(seeds)) {
  sample_size <- 15

  set.seed(seeds[i])
  sampled_indices <- sample(nrow(df1), sample_size, replace = FALSE)
  sampled_df <- df1[sampled_indices,]

  results_shapiro <- rstatix::shapiro_test(sampled_df$CN_mean)
  results_wilcox <- wilcox.test(df2$CN_mean - sampled_df$CN_mean)

  result_matrix[i, 1] <- results_shapiro$statistic
  result_matrix[i, 2] <- results_shapiro$p.value
  result_matrix[i, 3] <- results_wilcox$p.value
  result_matrix[i, 4] <- results_wilcox$statistic
}

write.csv(result_matrix, "result_matrix.csv")

```

OUTPUT DATA

```

ggsave(
  "cn_cell_state.jpeg",
  cn_cell_state,
  width = 8,
  height = 7,
  units = "in"
)

```

COMPARISON GENERAL EUKARYOTIC PRIMERS VS. SPECIFIC PRIMERS

STATISTICS

```

#Test for normality
rstatix::shapiro_test(df_primers$mean_concentration) -> normality

#Test for homogeneity of variance
rstatix::levene_test(df_primers, mean_concentration ~ Primers) -> variance

#Testing for significant difference between groups

rstatix::t_test(mean_concentration ~ Primers,
  data = df_primers,
  paired = T) -> sign_test

```

DATA VISUALIZATION

```
as.factor(df_primers$Primers) -> df_primers$Primers
relevel(df_primers$Primers, "Paramecium specific primers") -> df_primers$Primers

ggplot(df_primers, aes(x = Sample, y = mean_concentration, fill= Primers, color = Primers)) +
  geom_bar(stat = "identity", width = 0.75, position = position_dodge(width = 0.9)) +
  geom_errorbar(aes(ymin=mean_concentration-sd, ymax=mean_concentration+sd),
    position = position_dodge(0.9),
    width = 0.25,
    show.legend = FALSE) +
  xlab("single cells") +
  ylab("rRNA gene copy number per cell") +
  scale_y_continuous(labels = comma) +
  scale_fill_manual(values = palette_3) +
  scale_color_manual(values = palette_3) +
  my_theme +
  theme(
    legend.position = ("none"),
    legend.title = element_text(size = 12, colour = "black", face = "bold"),
    legend.text = element_text(size = 12, colour = "black")
  ) -> barplot_primers
```

OUTPUT DATA

```
ggsave(
  "barplot_primers.jpeg",
  barplot_primers,
  width = 9,
  height = 7,
  units = "in"
)
```

COMPARISON OF COPY NUMBER FOR INCREASING NUMBER OF CELLS

```
# Change str of df_scatter
as.factor(df_scatter$Technique) -> df_scatter$Technique
as.factor(df_scatter_all$Technique) -> df_scatter_all$Technique

#regression plot with numbers reduced and label 10^3

ggplot(df_scatter_all, aes(x=cellratio, y=copynumbers2)) +
  geom_point(alpha= 0.01) +
  #geom_pointrange(aes(ymin=copynumbers2-sd2, ymax=copynumbers2+sd2)) +
```

```

geom_smooth(method=lm, fullrange = T, color="black") +
stat_cor(p.accuracy = 0.001,
  aes(label = paste(..rr.label.., ..p.label.., sep = "~`,`~")),
  label.x = 0, label.y= 3600, show.legend = F) +
stat_regline_equation(label.x = 0, label.y = 3300)+
xlab("number of cells") +
ylab(bquote('total rRNA gene copy number '(x10^3))) +
my_theme +
theme(legend.position = "none",
  plot.title = element_text(
    size = 14,
    hjust = -0.1,
    vjust = 2
  )) -> cn_regression

cn_regression +
geom_point(data = df_scatter,
  mapping = aes(x = cellratio, y = copynumbers2)) +
geom_pointrange(data = df_scatter, aes(ymin = copynumbers2 - sd2, ymax =
  copynumbers2 + sd2)) -> scatter_plot

```

DATA VISUALIZATION

```

ggplot(df_ratio_reg, aes(x=valid_partitions, y=copynumber)) +
geom_point(color = "black")+
geom_smooth(method=lm, fullrange = F, color = "black") +
stat_cor(method = "pearson",
  aes(label = paste(..r.label.., ..p.label.., sep = "~`,`~")),
  show.legend = F) +
xlab("Valid partitions") +
ylab("log10 rRNA gene copy number per µl") +
scale_y_continuous(limits = c(10 , 2300), trans = "log10" ) +
my_theme +
theme(legend.position = "none",
  plot.title = element_text(
    size = 14,
    hjust = -0.1,
    vjust = 2
  ))-> corr_graph

```

OUTPUT DATA

```

ggsave(
  "scatterplot_cellnumbers_conf.jpeg",
  scatter_plot,
  width = 10,
  height = 7,
  units = "in"
)

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

```
ggsave(  
  "corr_graph_copies_partitions.jpeg",  
  corr_graph,  
  width = 10,  
  height = 8,  
  units = "in"  
)
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```

SESSION INFO

```
sessionInfo()
```

```
## R version 4.3.1 (2023-06-16)  
## Platform: aarch64-apple-darwin20 (64-bit)  
## Running under: macOS Ventura 13.0  
##  
## Matrix products: default  
## BLAS: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0.dylib  
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack.dylib; LAPACK v  
##  
## locale:  
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8  
##  
## time zone: Europe/Berlin  
## tzcode source: internal  
##  
## attached base packages:  
## [1] stats graphics grDevices utils datasets methods base  
##  
## other attached packages:  
## [1] scales_1.2.1 vegan_2.6-4 lattice_0.21-8 permute_0.9-7  
## [5] rstatix_0.7.2 ggpubr_0.6.0 knitr_1.44 lubridate_1.9.3  
## [9] forcats_1.0.0 stringr_1.5.0 dplyr_1.1.3 purrr_1.0.2  
## [13] readr_2.1.4 tidyr_1.3.0 tibble_3.2.1 ggplot2_3.4.3  
## [17] tidyverse_2.0.0  
##  
## loaded via a namespace (and not attached):  
## [1] gtable_0.3.4 xfun_0.40 coin_1.4-3 tzdb_0.4.0  
## [5] vctrs_0.6.3 tools_4.3.1 generics_0.1.3 sandwich_3.0-2  
## [9] stats4_4.3.1 parallel_4.3.1 fansi_1.0.4 cluster_2.1.4  
## [13] pkgconfig_2.0.3 Matrix_1.5-4.1 lifecycle_1.0.3 farver_2.1.1  
## [17] compiler_4.3.1 textshaping_0.3.6 munsell_0.5.0 codetools_0.2-19  
## [21] carData_3.0-5 htmltools_0.5.6 yaml_2.3.7 pillar_1.9.0  
## [25] car_3.1-2 MASS_7.3-60 multcomp_1.4-25 abind_1.4-5  
## [29] nlme_3.1-162 tidyselect_1.2.0 digest_0.6.33 mvtnorm_1.2-3
```

```
## [33] stringi_1.7.12    labeling_0.4.3    splines_4.3.1    fastmap_1.1.1
## [37] grid_4.3.1        colorspace_2.1-0  cli_3.6.1        magrittr_2.0.3
## [41] survival_3.5-5    utf8_1.2.3        TH.data_1.1-2    broom_1.0.5
## [45] libcoin_1.0-10    withr_2.5.1        backports_1.4.1  timechange_0.2.0
## [49] rmarkdown_2.25    matrixStats_1.0.0 ggsignif_0.6.4    ragg_1.2.5
## [53] zoo_1.8-12        modeltools_0.2-23 hms_1.1.3         evaluate_0.22
## [57] mgcv_1.8-42       rlang_1.1.1       polynom_1.4-1     glue_1.6.2
## [61] rstudioapi_0.15.0 R6_2.5.1          systemfonts_1.0.4
```

R.Version()

```
## $platform
## [1] "aarch64-apple-darwin20"
##
## $arch
## [1] "aarch64"
##
## $os
## [1] "darwin20"
##
## $system
## [1] "aarch64, darwin20"
##
## $status
## [1] ""
##
## $major
## [1] "4"
##
## $minor
## [1] "3.1"
##
## $year
## [1] "2023"
##
## $month
## [1] "06"
##
## $day
## [1] "16"
##
## $'svn rev'
## [1] "84548"
##
## $language
## [1] "R"
##
## $version.string
## [1] "R version 4.3.1 (2023-06-16)"
##
## $nickname
## [1] "Beagle Scouts"
```

```
knitr::write_bib(c(.packages()), "packages.bib")  
rm(list = ls())
```